# Senior Software Engineer — Take-Home Assignment

**Project: AgentHub**

## The scenario

Think of AgentHub as a **“Play Store” for AI agents**.

- A user lands on the platform and sees a **catalog of AI agents** — one for each profession (doctor, lawyer, accountant, recruiter, software engineer, teacher, etc.) — the same way they’d browse a list of apps in an app store.
- The user **picks one agent**.
- The user then **authenticates specifically for that agent**. That login is scoped to the agent they picked — it does not grant access to any other agent on the platform, the same way logging into one app doesn’t log you into an unrelated app.
- Once inside, the user **chats with that agent**, and the agent responds in character, powered by a real LLM.

We already have ~100 agents defined in `agents_sample.csv`. Your job is to build the platform that turns that CSV into a working, authenticated, chat-based product — and to design it so that agent #101, #500, or #10,000 is a config change, not a code change.

---

## Important clarification: what an “agent” actually is

**You are not building 100 different AI models or 100 different pieces of custom logic** — an agent here is persona/config data (including a system prompt) that gets passed to a real LLM API (OpenAI / Anthropic / Gemini / etc.) on each chat turn, not a bespoke AI you build yourself.

---

## User journey

A visitor browses the agent catalog, selects one agent, authenticates for that agent only, and chats with it — across desktop and mobile, on a live public URL, with no setup or explanation from you required.

## Admin / content pipeline journey (how agents get onto the platform)

Decide yourself.

---

## What “done” means

Partial credit exists, but a design doc with no working system is a fail.

### 1. Live, deployed, and usable by a stranger

- A real public URL (Vercel, Render, Railway, Fly.io, or your own infra — no `localhost`, no “run `npm install` first”)
- Loads and functions without you present to explain it
- Works on both desktop and mobile browser width

### 2. Real authentication — not mocked

- Signup and login work end to end: hashed/salted credentials, real sessions or JWTs, and protection against obvious attacks (no plaintext passwords, no secrets committed to the repo, HTTPS only)
- Login is **isolated per agent** — a test account created under one agent must not work under another. You choose the mechanism (fully separate systems vs. tenant-scoped shared auth) and must be able to defend the choice.

### 3. A working agent catalog and chat, backed by a real LLM

- A browsable catalog covering all ~100 provided agents
- At least **5 fully working agents**, each with a distinct persona/system prompt, actually calling a real LLM API and responding in character — not canned/hardcoded responses

- At least **3 of those 5 agents** must each expose **2–5 specialized sub-agents**, using a shared UI component but demonstrably different backend behavior per sub-agent (ask the same question to two sub-agents under different main agents and get answers reflecting different specialization)

### 4. Architecture that visibly isn't 100 copies of the same code

- Reviewers will look at your repo for evidence that **agents are configuration/data, not duplicated code paths**. If adding agent #101 requires writing new code instead of adding a config row, that's a fail on this criterion regardless of how polished the UI looks.

### 5. A real, if minimal, content pipeline

- Show how you'd take the raw CSV and turn it into working agent configs — a script is sufficient, no UI required. At minimum: given a new row in the CSV, produce a usable system prompt **without you hand-writing it**.

### 6. Source code and documentation

- Public repo (or a private one shared with us) with a README covering: setup, architecture decisions, what you'd do differently with more time, and known limitations
- Basic automated tests for at least the auth flow and the chat/config resolution logic — we're not expecting full coverage, but zero tests on a "production-grade" claim is a red flag

---

## Hard constraints

- **Timebox: 5 days.** Focused effort, not 24/7 — but the deployed system must exist and function by the deadline. No exceptions for "it works locally."
- **Backend must use a Python stack** (FastAPI or Django). Frontend framework, LLM provider, and hosting platform are your choice, and you should be ready to justify them.
- No paid infrastructure required beyond free tiers — budget constraints are part of the problem, not an excuse.
- If you use AI-assisted coding tools to build this (expected, even encouraged), be transparent about it in the README, including the name of the coding agent(s) you used.

## Stretch goals (not required, but noticed)

- An admin path to add a new agent via config/data entry, with zero code changes
- Basic rate limiting or cost controls on the LLM calls
- Observability — logs or a dashboard showing which agents are getting used
- CI that runs your tests on push

## What you submit

1. Live URL
2. Repo link
3. A test account per isolated-login boundary (or clear instructions to create one)